

Progetto Elementi di Ingegneria dei Linguaggi di Programmazione: Specifiche del Linguaggio C-ro

Russo Alberto - Prof: Gennaro Costagliola

1 Introduzione e Metodologia

Il presente documento definisce le specifiche per il linguaggio di programmazione **C-ro**, un compilatore basato sul linguaggio C ma caratterizzato da una sintassi e da parole chiave ispirate al dialetto napoletano. Come richiesto dalle specifiche di progetto, la documentazione copre le fasi di definizione lessicale, sintattica e semantica. La metodologia di implementazione prevede l'uso del linguaggio **Python** in combinazione con il parsing toolkit **Lark** per l'analisi lessicale e sintattica. Il codice target generato dal compilatore sarà codice **C** standard.

2 Specifiche Lessicali

L'analizzatore lessicale (lexer) è responsabile della scomposizione del codice sorgente in token. Utilizzando la sintassi delle espressioni regolari e delle stringhe letterali (come richiesto da Lark), vengono di seguito definiti i lessemi del linguaggio C-ro.

2.1 Parole Chiave (Keywords)

Le parole chiave del linguaggio sono riservate e non possono essere utilizzate come identificatori. Di seguito la mappatura tra i concetti standard del C e il dialetto C-ro:

Significato (C)	Keyword (C-ro)	Token
<i>Tipi di Dato e Costanti</i>		
int	numero	KW_INT
float	comm	KW_FLOAT
char	carattere	KW_CHAR
bool	bull	KW_BOOL
void	a_vacant	KW_VOID
const	const	KW_CONST
true	over	KW_TRUE
false	favz	KW_FALSE
<i>Strutture di Controllo e Cicli</i>		
if	si	KW_IF
else if	else if	KW_ELSEIF
else	s'invece	KW_ELSE
switch	facimm	KW_SWITCH
case	cas	KW_CASE
default	default	KW_DEFAULT
while	trament	KW_WHILE

Significato (C)	Keyword (C-ro)	Token
for	ppe	KW_FOR
do	faje	KW_DO
break	spacc	KW_BREAK
continue	vaje	KW_CONTINUE
<i>Funzioni e Input/Output</i>		
main	capo	KW_MAIN
function	funzion	KW_FUNCTION
return	torn	KW_RETURN
printf	stambf	KW_PRINTF
scanf	scanef	KW_SCANF
<i>Operatori Logici</i>		
and	e	KW_AND
or	o	KW_OR
not	non	KW_NOT

2.2 Identificatori e Costanti

Oltre alle parole chiave, il lexer riconosce i seguenti elementi definiti tramite espressioni regolari:

- **ID**: `/[a-zA-Z_][a-zA-Z0-9_]*/` (Es. nomi di variabili o funzioni).
- **INTEGER_CONST**: `/d+/` (Costanti intere).
- **FLOAT_CONST**: `/d+.`
`d+/` (Costanti in virgola mobile).
- **STRING_CONST**: `/"([^\n\r"]|\\[ \t\n\r"])*"/` (Stringhe racchiuse tra doppi apici).

I commenti e gli spazi bianchi (*whitespace*) verranno ignorati silenziosamente dall'analizzatore lessicale.

3 Specifiche Sintattiche

L'analizzatore sintattico (Parser) del compilatore **C-ro** elabora i token forniti dal lexer per costruire l'albero di parsing (Concrete Syntax Tree) e, successivamente, l'Albero Sintattico Astratto (AST).

La grammatica è definita utilizzando il metalinguaggio EBNF esteso fornito dal toolkit **Lark**. Le regole sintattiche, indicate in minuscolo, si avvalgono delle parole chiave in dialetto napoletano definite al passo precedente pur mantenendo intatta l'architettura dei costrutti del linguaggio C. Il parser supporta nativamente gli algoritmi *Earley* o *LALR(1)* per la risoluzione della grammatica.

3.1 Programma e Dichiarazioni

Un programma in C-ro è costituito da una sequenza di dichiarazioni di variabili e definizioni di funzioni, tra cui spicca in modo speciale il `main` (chiamato `capo`).

```
program: (var_decl | function_decl)*

// Dichiarazione di variabili (es. numero x = 5;)
var_decl: type ID (ASSIGN expr)? SEMI

// Dichiarazione di funzioni: 'funzion' tipo nome ( parametri ) { corpo }
function_decl: KW_FUNCTION type ID LPAR params? RPAR block
               | type KW_MAIN LPAR RPAR block

params: param (COMMA param)*
param: type ID

type: KW_INT | KW_FLOAT | KW_CHAR | KW_BOOL | KW_VOID
     | KW_CONST type
```

3.2 Blocchi e Istruzioni (Statements)

Rispettando i dettami del C standard, le istruzioni semplici terminano con il punto e virgola (SEMI), mentre le istruzioni composte sono racchiuse in blocchi { }.

```
block: LBRACE statement* RBRACE

statement: var_decl
          | if_stmt
          | switch_stmt
          | while_stmt
          | for_stmt
          | do_while_stmt
          | return_stmt
          | break_stmt
          | continue_stmt
          | expr SEMI
          | block
```

3.3 Strutture di Controllo e Cicli

La sintassi per la selezione e l'iterazione riflette fedelmente il C, con l'innesto delle keyword in dialetto. Si noti il supporto al "dangling else" per i costrutti opzionali `else if` e `s'invece`.

```
// Costrutti si, else if, s'invece
if_stmt: KW_IF LPAR expr RPAR statement
        (KW_ELSEIF LPAR expr RPAR statement)* (KW_ELSE statement)?

// Costrutto switch (facimm) e case (cas)
switch_stmt: KW_SWITCH LPAR expr RPAR LBRACE case_stmt* default_stmt? RBRACE
case_stmt: KW_CASE expr COLON statement*
default_stmt: KW_DEFAULT COLON statement*
```

```

// Cicli: while (trament), for (ppe), do-while (faje ... trament)
while_stmt: KW_WHILE LPAR expr RPAR statement

for_stmt: KW_FOR LPAR (var_decl | expr SEMI | SEMI) expr? SEMI expr? RPAR statement

do_while_stmt: KW_DO statement KW_WHILE LPAR expr RPAR SEMI

// Istruzioni di salto
return_stmt: KW_RETURN expr? SEMI
break_stmt: KW_BREAK SEMI
continue_stmt: KW_CONTINUE SEMI

```

3.4 Espressioni e Precedenza degli Operatori

Per risolvere le ambiguità sulle precedenze matematiche e logiche, la grammatica C-ro adotta una struttura "a strati" (stratified grammar), sfruttando l'inlining di Lark (prefisso ?) per snellire l'AST generato. Gli operatori logici e, o, non sostituiscono le versioni standard C.

```

?expr: assign_expr

?assign_expr: logical_or_expr (ASSIGN assign_expr)?

?logical_or_expr: logical_and_expr (KW_OR logical_and_expr)*

?logical_and_expr: equality_expr (KW_AND equality_expr)*

?equality_expr: relational_expr ((EQ | NEQ) relational_expr)*

?relational_expr: add_expr ((LT | LE | GT | GE) add_expr)*

?add_expr: mul_expr ((PLUS | MINUS) mul_expr)*

?mul_expr: unary_expr ((STAR | SLASH | MOD) unary_expr)*

?unary_expr: (KW_NOT | MINUS) unary_expr
             | primary_expr

?primary_expr: ID
              | INT_CONST | FLOAT_CONST | STRING_CONST | CHAR_CONST
              | KW_TRUE | KW_FALSE
              | LPAR expr RPAR
              | function_call

// Chiamate a funzione, incluse le primitive di I/O (stambrf, scanf)
function_call: (KW_PRINTF | KW_SCANF | ID) LPAR args? RPAR
args: expr (COMMA expr)*

```

4 Analisi Semantica

L'analisi semantica del compilatore **C-ro** ha lo scopo di verificare che un programma, seppur sintatticamente corretto, sia coerente con le regole logiche e di tipo del linguaggio. Tale fase

viene implementata attraversando l'Albero Sintattico Astratto (AST) [3] e si articola in due componenti principali: la Gestione dello Scoping e il Type Checking [1].

4.1 Gestione dello Scoping e Tabella dei Simboli

Per risolvere la visibilità delle variabili e delle funzioni, il compilatore crea e aggiorna dinamicamente un Type Environment (Tabella dei Simboli). A livello implementativo, si utilizza uno stack di tabelle hash [4]. Ogni qualvolta il parser incontra un costrutto che apre un nuovo blocco logico (la funzione **capo**, una normale **funzion**, i cicli **trament** / **ppe**, o i costrutti selettivi **si** / **facimm**), viene creata una nuova tabella dei simboli agganciata allo scope corrente, che mantiene un riferimento al suo blocco "padre" [5], [6].

Le regole di validazione dello scope sono le seguenti:

- **Dichiarazione (Scrittura):** **SE** l'analizzatore incontra la dichiarazione di un identificatore (es. una variabile di tipo **numero**), verifica prima la tabella nello scope locale. **SE** l'identificatore è già presente nello scope corrente, **ALLORA** genera un errore di "dichiarazione multipla" [7]. Altrimenti, memorizza l'ID e il suo tipo nella tabella.
- **Risoluzione (Lookup):** **SE** il codice fa uso di una variabile, l'analizzatore la cerca nella tabella corrente [8]. **SE** non la trova, risale la pila fino alla tabella dello scope globale. **SE** raggiunge la tabella globale senza trovare la dichiarazione, **ALLORA** lancia un errore di "identificatore non dichiarato" [8].

4.2 Type System e Regole d'Inferenza (Type Checking)

Il sistema di tipi assicura la corretta composizione delle espressioni tramite regole d'inferenza. Utilizzando il Type Environment T , ogni nodo dell'AST riceve un tipo specifico per garantirne la validità formale [9], [2]. Di seguito le principali regole d'inferenza in formato **IF-THEN** per i costrutti del linguaggio **C-ro**:

Regole per le Operazioni Matematiche (es. Addizione +):

- **IF** l'operando A è di tipo **numero** in T **AND** l'operando B è di tipo **numero** in T , **THEN** la loro somma è semanticamente corretta e il risultato ha tipo **numero** [2].
- *Type Promotion:* **IF** l'operando A è di tipo **numero** **AND** l'operando B è di tipo **comm** (float), **THEN** l'operazione è valida e il compilatore promuove il risultato al tipo **comm** per prevenire perdite di precisione [10], [11].

Regola per le Operazioni Logiche (e, o, non):

- **IF** i tipi dei nodi figli dell'operatore sono compatibili con il tipo **bull** (booleano) in T , **THEN** l'espressione è senza errori ed il suo tipo finale è **bull** [12].

Regola per l'Assegnazione (=):

- **IF** l'identificatore a sinistra è stato dichiarato con un tipo t in T **AND** l'espressione valutata a destra restituisce un valore compatibile con il tipo t , **THEN** l'assegnazione è semanticamente corretta (e il nodo assume il tipo **a_vacant** / **NOTYPE**) [13]. **ELSE** restituisce un errore di "Type Mismatch" [2].

Regole per i Costrutti di Controllo e Cicli (si, else if, s'invece, trament, ppe):

- **IF** l'espressione di guardia (condizione) tra parentesi restituisce un tipo **bull** **AND** i blocchi di istruzioni annidati non presentano errori semantici propri, **THEN** la struttura di controllo è correttamente tipata [13].

Regola per le Chiamate a Funzione (funzion):

- **IF** la funzione invocata è presente in T con argomenti formali di tipo $t_1, t_2, \dots, t_n$ e tipo di ritorno t_{ret} , **AND** gli argomenti effettivi passati (parametri in input) sono rigorosamente compatibili con la firma posizionale $(t_1, t_2, \dots, t_n)$, **THEN** la chiamata a funzione è valida e il suo tipo nell'AST viene risolto come t_{ret} [2].

5 Generazione del Codice

L'ultima fase del compilatore **C-ro** prevede la trasformazione dell'Albero Sintattico Astratto (AST), ormai validato semanticamente, in codice sorgente C standard. Per effettuare questa traduzione, il compilatore adotta il **Visitor Pattern con Riflessione** in linguaggio Python [1, 2]. A differenza del pattern classico basato sul *double dispatch* (tipico di linguaggi come Java, che richiede il metodo `accept()` in ogni nodo), l'approccio riflessivo in Python risulta meno invasivo e riduce il codice *boilerplate* [4, 5]. I nodi dell'AST fungono esclusivamente da contenitori di dati, mentre tutta la logica di traduzione è concentrata in un'unica classe *Visitor* [3, 6].

5.1 Struttura dei Nodi AST

I nodi dell'AST sono implementati come semplici classi base prive di metodi logici. Ad esempio, una dichiarazione di variabile o un'operazione matematica conterranno solo le informazioni essenziali [6].

```
class ASTNode:
    pass

class VarDeclNode(ASTNode):
    def __init__(self, var_type, name, value):
        self.var_type = var_type # Es. 'numero'
        self.name = name        # Es. 'x'
        self.value = value      # Nodo espressione

class PrintNode(ASTNode):
    def __init__(self, expr):
        self.expr = expr        # Quello che segue 'stamf'
```

5.2 Implementazione del CodeGeneratorVisitor

Il generatore di codice è una classe che implementa un metodo generico `visit(self, node)`. Sfruttando la funzione nativa `getattr` di Python, il visitor ricava dinamicamente il nome della classe del nodo analizzato (es. `AddNode`) e invoca automaticamente il metodo di traduzione corrispondente (es. `visit_AddNode`) [7, 8]. Di seguito si mostra lo scheletro del traduttore che mappa le keyword napoletane nei costrutti nativi del C:

```
class CodeGeneratorVisitor:
    def visit(self, node):
        """Metodo generico che delega la chiamata usando la riflessione"""
        class_name = node.__class__.__name__
        method_name = f'visit_{class_name}'
        method = getattr(self, method_name, self.generic_visit)
        return method(node)

    def generic_visit(self, node):
```

```

        raise Exception(f"Nessun metodo {node.__class__.__name__} trovato!")

def visit_ProgramNode(self, node):
    """Visita la radice del programma e include le librerie standard"""
    code = "#include <stdio.h>\n#include <stdbool.h>\n\n"
    for child in node.children:
        code += self.visit(child) + "\n"
    return code

def visit_MainFuncNode(self, node):
    """Traduce la keyword 'capo' nel main del C"""
    body_code = self.visit(node.body)
    return f"int main() {{\n{body_code}\n    return 0;\n}}"

def visit_VarDeclNode(self, node):
    """Mappa i tipi (es. numero -> int, comm -> float, bull -> bool)"""
    type_mapping = {
        'numero': 'int',
        'comm': 'float',
        'carattere': 'char',
        'bull': 'bool'
    }
    c_type = type_mapping.get(node.var_type, 'void')
    val_code = self.visit(node.value) if node.value else "0"
    return f"{c_type} {node.name} = {val_code};"

def visit_IfNode(self, node):
    """Traduce i costrutti selettivi (si, else if, s'invece)"""
    cond = self.visit(node.condition)
    body = self.visit(node.body)
    code = f"if ({cond}) {{\n{body}\n}}"

    for elif_node in node.elifs:
        # Traduce i blocchi 'else if'
        el_cond = self.visit(elif_node.condition)
        el_body = self.visit(elif_node.body)
        code += f" else if ({el_cond}) {{\n{el_body}\n}}"

    if node.else_body:
        el_body = self.visit(node.else_body)
        code += f" else {{\n{el_body}\n}}"

    return code

def visit_WhileNode(self, node):
    """Traduce il costrutto iterativo 'trament' in un while C"""
    cond = self.visit(node.condition)
    body = self.visit(node.body)
    return f"while ({cond}) {{\n{body}\n}}"

def visit_PrintNode(self, node):

```

```

"""Traduce la keyword 'stambf' in printf"""
expr = self.visit(node.expr)
# Nota: in un compilatore completo il formato verrebbe inferito dal tipo
return f'printf("%d\\n", {expr});'

```

6 Note di Implementazione e Scelte Progettuali

Questa sezione descrive le scelte tecniche effettuate durante la fase di implementazione del compilatore, evidenziando in particolare i punti in cui l'implementazione concreta ha richiesto adattamenti rispetto alla progettazione descritta nelle sezioni precedenti, dovuti a vincoli pratici del toolkit **Lark** o a esigenze di robustezza emerse durante lo sviluppo.

6.1 Scelta dell'Algoritmo di Parsing

Sebbene la grammatica supporti nativamente sia l'algoritmo *Earley* sia *LALR(1)*, l'implementazione finale utilizza **Earley** con risoluzione automatica delle ambiguità (`ambiguity='resolve'`). La scelta è motivata dalla presenza di alcune ambiguità locali nella grammatica, in particolare il classico problema del *dangling else* nel costrutto `if_stmt` (un `s'invece` può sintatticamente appartenere sia al `si` più esterno che a quello più interno, in caso di `si` annidati privi di blocchi). Earley gestisce nativamente le grammatiche ambigue, risolvendo il *dangling else* nel modo standard (associando l'`s'invece` al `si` più vicino) senza richiedere una riscrittura della grammatica per eliminare il conflitto, come sarebbe stato invece necessario adottando *LALR(1)*.

6.2 Adattamento della Grammatica ai Vincoli di Lark

Lark non supporta la sintassi a gruppi ripetuti contenenti sequenze multi-simbolo all'interno della stessa regola: ad esempio `(KW_ELSEIF LPAR expr RPAR statement)*`, così come scritto nella Sezione 3.3, genera un conflitto interno (*"Rules defined twice"*) quando combinato con produzioni opzionali adiacenti. È stato quindi necessario introdurre delle **regole ausiliarie** per ciascun gruppo ripetuto multi-simbolo:

```

if_stmt: "si" "(" expr ")" statement elif_clause* else_clause?
elif_clause: "else" "if" "(" expr ")" statement
else_clause: "s'invece" statement

```

La stessa tecnica è stata applicata al costrutto `for_stmt`, introducendo una regola `for_init` per gestire le tre alternative dell'inizializzatore (dichiarazione di variabile, espressione, oppure vuoto). Una conseguenza di questa scelta è che il **Transformer** (classe `CroTransformer`) deve gestire esplicitamente alcuni nodi "di passaggio" che Lark genera per le regole prive di alias `-> nome`: ad esempio `unary_expr` e `primary_expr`, quando vengono risolte tramite l'alternativa priva di etichetta, sono comunque incapsulate da Lark in un nodo **Tree** con il nome della regola stessa. Il Transformer implementa quindi dei metodi "passthrough" (es. `def unary_expr(self, items): return items[0]`) che si limitano a restituire il figlio già trasformato, appiattendolo questi nodi intermedi nell'AST finale.

6.3 Operatore di Indirizzo (&) per l'Input

La grammatica originale (Sezione 3.4) non prevedeva un operatore esplicito di indirizzo. In fase di implementazione, per consentire un utilizzo realistico della primitiva `scanef` — che, mappando direttamente sulla funzione `scanf` del C, richiede il passaggio per indirizzo della variabile di destinazione — è stato introdotto l'operatore unario `&`, applicabile esclusivamente a un identificatore:


```

unary_expr: "non" unary_expr -> not_expr
           | "-" unary_expr -> neg_expr
           | "&" ID          -> addr_of
           | primary_expr

```

L'analisi semantica impone due vincoli aggiuntivi su questo operatore, assenti nella specifica originale: l'identificatore a cui si applica deve essere stato dichiarato (altrimenti errore di "identificatore non dichiarato"), e non può essere una costante `const` (altrimenti errore di "indirizzo di costante"). Inoltre, gli argomenti di `scanf` successivi alla stringa di formato devono obbligatoriamente essere espressioni `&variabile`: un argomento privo di `&` viene rifiutato in fase di analisi semantica, in analogia con il corretto uso di `scanf` in C.

6.4 Implementazione dello Scoping

La gestione dello scoping descritta nella Sezione 4.1 è stata implementata tramite una catena di oggetti `SymbolTable`, ciascuno con un riferimento esplicito alla propria tabella `parent` (anziché tramite uno stack di tabelle hash indipendenti). Rispetto a quanto indicato nella specifica originale — che limita l'apertura di un nuovo scope ai costrutti `capo`, `funzion`, `trament`, `ppe`, `si` e `facimm` — l'implementazione generalizza la regola: **ogni blocco { } (nodo `BlockNode`) apre un nuovo scope**, indipendentemente dal costrutto che lo contiene. Questa scelta è più conservativa e coerente con la semantica del C standard, dove anche un blocco "nudo" introduce un nuovo scope, e consente ad esempio di dichiarare due variabili omonime in due rami `si/s'invece` distinti senza generare conflitti. Fa eccezione il corpo diretto di una funzione: lo scope dei parametri formali e quello del blocco del corpo della funzione vengono fusi in un unico scope, in modo che un parametro e una variabile locale di primo livello condividano naturalmente lo stesso ambiente, evitando un livello di annidamento ridondante.

6.5 Gestione degli Errori

A differenza di un'implementazione che si arresta al primo errore semantico riscontrato, l'analizzatore semantico (classe `SemanticAnalyzer`) accumula **tutti** gli errori rilevati durante un'unica visita dell'AST in una lista, riportandoli congiuntamente al termine dell'analisi. Questa scelta offre un feedback più completo allo sviluppatore del programma C-ro, in linea con il comportamento dei compilatori di produzione (es. `gcc`), che tipicamente non si arrestano al primo errore incontrato.

6.6 Generazione del Codice: Indentazione Relativa

Diversamente dallo scheletro presentato in Sezione 5.2, dove l'indentazione è gestita implicitamente tramite stringhe letterali `"\n"` fisse, l'implementazione finale del `CodeGenerator` adotta uno schema di **indentazione relativa**: ogni metodo che genera codice multi-riga produce una stringa la cui prima riga si trova a colonna 0, mentre il contenuto interno (es. le istruzioni di un blocco) viene indentato di un solo livello tramite una funzione ausiliaria. Quando questa stringa viene inserita in un contesto già indentato (es. un `si` dentro un `trament` dentro il corpo di `capo`), l'intero blocco generato — comprese le righe già indentate al suo interno — viene spostato uniformemente di un ulteriore livello dal chiamante. Questo approccio puramente relativo, privo di un contatore di profondità condiviso a livello globale, evita un bug di raddoppio dell'indentazione che si presenterebbe naturalmente nei costrutti annidati su più livelli (es. uno `switch` dentro un `for` dentro `capo`) se si utilizzasse un singolo contatore incrementato e decrementato manualmente ad ogni livello di annidamento.

6.7 Rappresentazione dei Nodi AST

I nodi dell'AST sono stati implementati come `dataclass` di Python (decoratore `@dataclass`) anziché come classi con costruttore `__init__` scritto manualmente, come mostrato a titolo di esempio nella Sezione 5.1. Questa scelta riduce il codice *boilerplate* (costruttore, `__repr__` e confronto di uguaglianza generati automaticamente) mantenendo inalterata la filosofia dei nodi come semplici contenitori di dati privi di logica propria.

6.8 Validazione e Test

La correttezza del compilatore è stata validata tramite una suite di test automatizzati (`test_suite.py`) composta da test unitari di compilazione — uno per ciascun costrutto del linguaggio, inclusi i casi di errore sintattico e semantico attesi — e da test di integrazione end-to-end, che compilano il codice C generato con `gcc -Wall -Wextra` e ne eseguono il binario fornendo input simulato su standard input, verificando l'output prodotto a runtime. Fra questi ultimi è incluso il programma dimostrativo `calcolatrice.cro`: un'applicazione a menu interattivo che esercita la quasi totalità dei costrutti del linguaggio (funzioni, selezione, iterazione, I/O, operatori logici e relazionali, costanti). È inoltre presente un file di test negativo, `errore.cro`, contenente nove errori semantici intenzionali (uno per ciascuna categoria controllata dall'analizzatore), utile a verificare e dimostrare che il compilatore individua correttamente tutti gli errori in un'unica passata.

6.9 Limitazioni Note

Il linguaggio C-ro, così come implementato, non supporta: array, struct, puntatori generici (l'operatore `&` è utilizzabile esclusivamente come argomento di `scanf`), né gestione esplicita della memoria dinamica. Ogni funzione restituisce un singolo valore, in coerenza con la semantica nativa del linguaggio C target.